

Practical – 2

Practical 2 AIM:

AIM: Hadoop Installation, Configuration and HDFS Operations on Windows/Linux.

Problem Definition :

You have recently joined DataAnalytics Inc. as a Hadoop Administrator. The organization processes large-scale customer, sales, and marketing data using the Hadoop ecosystem. Due to increasing data volumes, the company has decided to migrate its data storage infrastructure from traditional file systems to Hadoop Distributed File System (HDFS). As a Hadoop Administrator, your responsibility is to install and configure Hadoop in a Single-Node Cluster environment on Windows/Linux, verify the Hadoop services, manage HDFS storage, and perform data management operations. Management wants a fully functional Hadoop environment that can store, retrieve, and process big data efficiently. Your task is to install Hadoop, configure the Hadoop ecosystem, start Hadoop services, and perform HDFS operations using Hadoop commands.

Task:

- Hadoop Installation & Configuration
- Access Hadoop Web UI.
- 1. Create directories in HDFS.
- 2. Upload files to HDFS.
- 3. List HDFS files and directories.
- 4. Display file contents.
- 5. Copy files within HDFS.
- 6. Move files within HDFS.
- 7. Rename files.
- 8. Download files from HDFS.
- 9. Delete files and directories.
- 10. Check HDFS report.
- 11. Verify block information.
- 12. Check Hadoop cluster health.
- 13. Monitor Hadoop processes.
- 14. Check storage utilization.

Output:

Steps to install

Check Hadoop Version

```
hadoop version
```

Check HDFS Version

```
hdfs version
```

Check HADOOP_HOME

```
echo $HADOOP_HOME
```

Check HDFS Home

```
echo $HADOOP_HDFS_HOME
```

```
# Find Hadoop Installation Location  
which hadoop
```

```
# Find HDFS Command Location  
which hdfs
```

```
# Start Hadoop  
start-dfs.sh  
start-yarn.sh
```

```
# Check Running Hadoop Processes  
jps
```

```
# Create HDFS Directories  
hdfs dfs -mkdir /project  
hdfs dfs -mkdir /project/data
```

```
# Create a Sample File  
echo "Hello Hadoop" > sample.txt
```

```
# Upload File to HDFS  
hdfs dfs -put sample.txt /project/data
```

```
# List HDFS Files and Directories  
hdfs dfs -ls /  
hdfs dfs -ls /project  
hdfs dfs -ls /project/data
```

```
# Display File Contents  
hdfs dfs -cat /project/data/sample.txt
```

```
# Copy File Within HDFS  
hdfs dfs -cp /project/data/sample.txt /project/sample_copy.txt
```

```
# Move File Within HDFS  
hdfs dfs -mv /project/sample_copy.txt /project/data/
```

```
# Rename File  
hdfs dfs -mv /project/data/sample.txt /project/data/newfile.txt
```

```
# Download File from HDFS  
hdfs dfs -get /project/data/newfile.txt
```

```
# Delete File  
hdfs dfs -rm /project/data/newfile.txt
```

Delete Directory

hdfs dfs -rm -r /project

Check HDFS Report

hdfs dfsadmin -report

Verify Block Information

hdfs fsck / -files -blocks

Check Hadoop Cluster Health

hdfs fsck /

Monitor Hadoop Processes

jps

Check HDFS Storage Utilization

hdfs dfs -df -h

Check Linux Disk Usage

df -h

Check Directory Size

du -sh *

Stop Hadoop

stop-yarn.sh

stop-dfs.sh

```

bhumi@kali: ~
Session Actions Edit View Help

(bhumi@kali)-[~]
└─$ hadoop version
Hadoop 3.3.6
Source code repository https://github.com/apache/hadoop.git -r 1be78238728da9266a4f88195058f08fd012bf9c
Compiled by ubuntu on 2023-06-18T08:22Z
Compiled on platform linux-x86_64
Compiled with protoc 3.7.1
From source with checksum 5652179ad55f76cb287d9c633bb53bbd
This command was run using /usr/local/hadoop/share/hadoop/common/hadoop-common-3.3.6.jar

(bhumi@kali)-[~]
└─$ start-dfs.sh
Starting namenodes on [localhost]
localhost: namenode is running as process 43989. Stop it first and ensure /tmp/hadoop-kali-namenode.pid file is empty
Starting datanodes
localhost: datanode is running as process 44091. Stop it first and ensure /tmp/hadoop-kali-datanode.pid file is empty
Starting secondary namenodes [kali]
kali: secondarynamenode is running as process 44274. Stop it first and ensure /tmp/hadoop-kali-secondarynamenode.pid

(bhumi@kali)-[~]
└─$ start-yarn.sh
Starting resourcemanager
resourcemanager is running as process 45357. Stop it first and ensure /tmp/hadoop-kali-resourcemanager.pid file is em
Starting nodemanagers
localhost: nodemanager is running as process 45458. Stop it first and ensure /tmp/hadoop-kali-nodemanager.pid file is

(bhumi@kali)-[~]
└─$ jps
45458 NodeManager
44274 SecondaryNameNode
83749 Jps
43989 NameNode
44091 DataNode
45357 ResourceManager

(bhumi@kali)-[~]
└─$ hdfs dfs -ls /
Found 2 items
drwxr-xr-x - kali supergroup          0 2026-08-08 14:43 /tmp
drwxr-xr-x - kali supergroup          0 2026-08-08 14:30 /user

(bhumi@kali)-[~]
└─$ hdfs dfs -mkdir -p /data/customer
hdfs dfs -mkdir -p /data/sales
hdfs dfs -mkdir -p /data/marketing

```

```

bhumi@kali: ~
Session Actions Edit View Help

hdfs dfs -mkdir -p /data/marketing

(bhumi@kali)-[~]
└─$ hdfs dfs -ls -R /data
drwxr-xr-x - kali supergroup          0 2026-08-08 15:29 /data/customer
drwxr-xr-x - kali supergroup          0 2026-08-08 15:29 /data/marketing
drwxr-xr-x - kali supergroup          0 2026-08-08 15:29 /data/sales

(bhumi@kali)-[~]
└─$ echo "CustomerID Name Amount" > customer.txt
echo "101 Riddhi 5000" >> customer.txt
echo "102 Tanisha 7000" >> customer.txt

(bhumi@kali)-[~]
└─$ cat customer.txt
CustomerID Name Amount
101 Riddhi 5000
102 Tanisha 7000

(bhumi@kali)-[~]
└─$ hdfs dfs -put customer.txt /data/customer/

(bhumi@kali)-[~]
└─$ hdfs dfs -ls /data/customer
Found 1 items
-rw-r--r-- 1 kali supergroup          56 2026-08-08 15:29 /data/customer/customer.txt

(bhumi@kali)-[~]
└─$ hdfs dfs -cat /data/customer/customer.txt
CustomerID Name Amount
101 Riddhi 5000
102 Tanisha 7000

(bhumi@kali)-[~]
└─$ hdfs dfs -cp /data/customer/customer.txt /data/sales/

(bhumi@kali)-[~]
└─$ hdfs dfs -mv /data/sales/customer.txt /data/marketing/

(bhumi@kali)-[~]
└─$ hdfs dfs -mv /data/marketing/customer.txt /data/marketing/customer_data.txt

(bhumi@kali)-[~]
└─$ mkdir -p ~/hadoop_download
hdfs dfs -get /data/marketing/customer_data.txt ~/hadoop_download/

```

```

Session Actions Edit View Help
(bhumi@kali)-[~]
└─$ mkdir -p ~/hadoop_download
hdfs dfs -get /data/marketing/customer_data.txt ~/hadoop_download/

(bhumi@kali)-[~]
└─$ hdfs dfsadmin -report
Configured Capacity: 84053143552 (78.28 GB)
Present Capacity: 58131976326 (54.14 GB)
DFS Remaining: 58131378176 (54.14 GB)
DFS Used: 598150 (584.13 KB)
DFS Used%: 0.00%
Replicated Blocks:
    Under replicated blocks: 2
    Blocks with corrupt replicas: 0
    Missing blocks: 0
    Missing blocks (with replication factor 1): 0
    Low redundancy blocks with highest priority to recover: 2
    Pending deletion blocks: 0
Erasure Coded Block Groups:
    Low redundancy block groups: 0
    Block groups with corrupt internal blocks: 0
    Missing block groups: 0
    Low redundancy blocks with highest priority to recover: 0
    Pending deletion blocks: 0

Live datanodes (1):

Name: 127.0.0.1:9866 (localhost)
Hostname: kali
Decommission Status : Normal
Configured Capacity: 84053143552 (78.28 GB)
DFS Used: 598150 (584.13 KB)
Non DFS Used: 21604446074 (20.12 GB)
DFS Remaining: 58131378176 (54.14 GB)
DFS Used%: 0.00%
DFS Remaining%: 69.16%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 0
Last contact: Sat Aug 08 15:31:06 EDT 2026
Last Block Report: Sat Aug 08 14:02:45 EDT 2026
Num of Blocks: 8

```

```

Session Actions Edit View Help
(bhumi@kali)-[~]
└─$ hdfs fsck /data/marketing/customer_data.txt -files -blocks -locations
Connecting to namenode via http://localhost:9870/fsck?ugi=kali&files=1&blocks=1&locations=1&path=%2Fdata%2Fmarketing%2Fcustomer_data.txt
FSCK started by kali (auth:SIMPLE) from /127.0.0.1 for path /data/marketing/customer_data.txt at Sat Aug 08 15:31:17 EDT 2026

/data/marketing/customer_data.txt 56 bytes, replicated: replication=1, 1 block(s): OK
0. BP-1633687240-127.0.0.1-1786212041311:blk_1073741832_1008 len=56 Live_repl=1 [DatanodeInfoWithStorage[127.0.0.1:9866]]

Status: HEALTHY
Number of data-nodes: 1
Number of racks: 1
Total dirs: 0
Total symlinks: 0

Replicated Blocks:
Total size: 56 B
Total files: 1
Total blocks (validated): 1 (avg. block size 56 B)
Minimally replicated blocks: 1 (100.0 %)
Over-replicated blocks: 0 (0.0 %)
Under-replicated blocks: 0 (0.0 %)
Mis-replicated blocks: 0 (0.0 %)
Default replication factor: 1
Average block replication: 1.0
Missing blocks: 0
Corrupt blocks: 0
Missing replicas: 0 (0.0 %)
Blocks queued for replication: 0

Erasure Coded Block Groups:
Total size: 0 B
Total files: 0
Total block groups (validated): 0
Minimally erasure-coded block groups: 0
Over-erasure-coded block groups: 0
Under-erasure-coded block groups: 0
Unsatisfactory placement block groups: 0
Average block group size: 0.0
Missing block groups: 0
Corrupt block groups: 0
Missing internal blocks: 0
Blocks queued for replication: 0
FSCK ended at Sat Aug 08 15:31:18 EDT 2026 in 78 milliseconds

```

```

bhumi@kali: ~
Session Actions Edit View Help

(bhumi@kali)-[~]
$ hdfs fsck /
Connecting to namenode via http://localhost:9870/fsck?ugi=kali&path=%2F
FSCK started by kali (auth:SIMPLE) from /127.0.0.1 for path / at Sat Aug 08 15:31:23 EDT 2026

/tmp/hadoop-yarn/staging/kali/.staging/job_1786212234441_0001/job.jar: Under replicated BP-1633687240-127.0.1.1-17862
ssioned replica(s), 0 decommissioning replica(s).

/tmp/hadoop-yarn/staging/kali/.staging/job_1786212234441_0001/job.split: Under replicated BP-1633687240-127.0.1.1-178
missioned replica(s), 0 decommissioning replica(s).

Status: HEALTHY
Number of data-nodes: 1
Number of racks: 1
Total dirs: 15
Total symlinks: 0

Replicated Blocks:
Total size: 517566 B
Total files: 8
Total blocks (validated): 8 (avg. block size 64695 B)
Minimally replicated blocks: 8 (100.0 %)
Over-replicated blocks: 0 (0.0 %)
Under-replicated blocks: 2 (25.0 %)
Mis-replicated blocks: 0 (0.0 %)
Default replication factor: 1
Average block replication: 1.0
Missing blocks: 0
Corrupt blocks: 0
Missing replicas: 18 (69.23077 %)
Blocks queued for replication: 0

Erasure Coded Block Groups:
Total size: 0 B
Total files: 0
Total block groups (validated): 0
Minimally erasure-coded block groups: 0
Over-erasure-coded block groups: 0
Under-erasure-coded block groups: 0
Unsatisfactory placement block groups: 0
Average block group size: 0.0
Missing block groups: 0
Corrupt block groups: 0
Missing internal blocks: 0

```

```

bhumi@kali: ~
Session Actions Edit View Help

(bhumi@kali)-[~]
$ hdfs dfsadmin -safemode get
Safe mode is OFF

(bhumi@kali)-[~]
$ jps
45458 NodeManager
44274 SecondaryNameNode
84453 Jps
43989 NameNode
44091 DataNode
45357 ResourceManager

(bhumi@kali)-[~]
$ yarn node -list
2026-08-08 15:31:48,478 INFO client.DefaultNoHARMAFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
Total Nodes:1
Node-Id Node-State Node-Http-Address Number-of-Running-Containers
kali:34631 RUNNING kali:8042 0

(bhumi@kali)-[~]
$ hdfs dfs -df -h
Filesystem Size Used Available Use%
hdfs://localhost:9000 78.3 G 600 K 54.1 G 0%

(bhumi@kali)-[~]
$ hdfs dfs -du -h /data
56 56 /data/customer
56 56 /data/marketing
0 0 /data/sales

(bhumi@kali)-[~]
$ hdfs dfs -rm /data/marketing/customer_data.txt
Deleted /data/marketing/customer_data.txt

(bhumi@kali)-[~]
$ hdfs dfs -rmdir /data/marketing
hdfs dfs -rmdir /data/sales
hdfs dfs -rmdir /data/customer
hdfs dfs -rmdir /data
rmdir: '/data/customer': Directory is not empty
rmdir: '/data': Directory is not empty

(bhumi@kali)-[~]
$

```

Key Questions / Analysis

Q1. Why is Hadoop preferred for Big Data storage?

Answer: Hadoop is preferred because it can store and process very large amounts of data across multiple computers with high scalability, fault tolerance, and low cost.

Q2. What is the role of NameNode and DataNode in HDFS?

Answer: • NameNode: Manages the file system metadata and keeps track of file locations. • DataNode: Stores the actual data blocks and serves read/write requests. D25DCS159 Big Data Analytics (CSUE301)

Q3. What is the difference between HDFS and traditional file systems?

Answer: • HDFS: Stores data across multiple machines, supports very large files, and provides fault tolerance. • Traditional file system: Stores data on a single machine with limited scalability and fault tolerance.

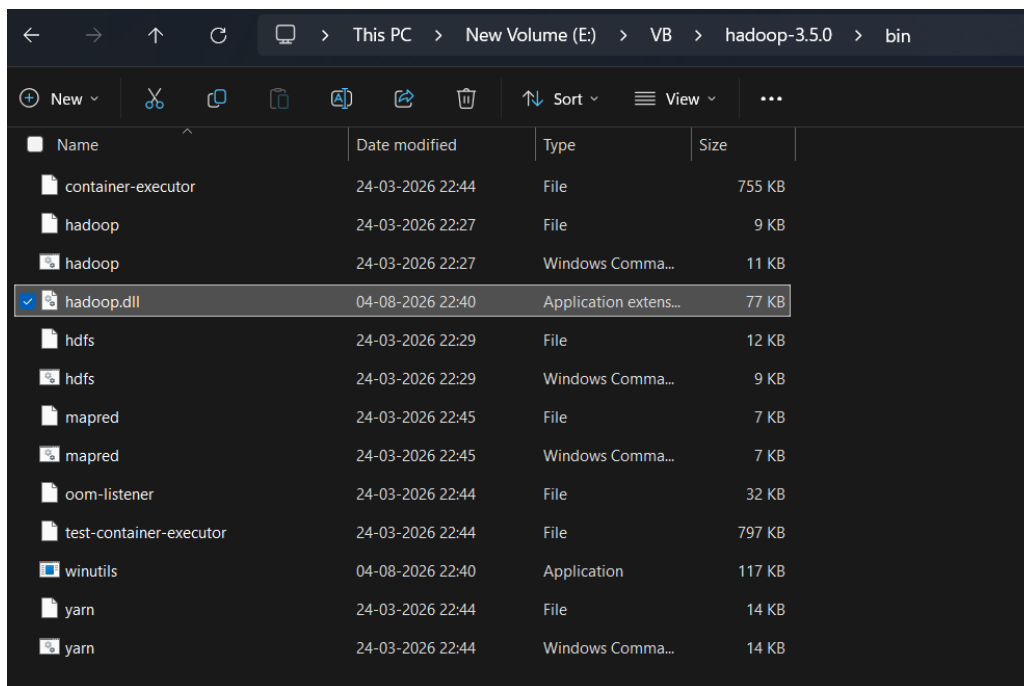
Q4. What are the advantages of distributed storage?

Answer: Distributed storage provides high scalability, fault tolerance, better reliability, improved performance through parallel access, and high availability of data.

Supplementary Problems

1. Configure Hadoop pseudo-distributed mode.
2. Change HDFS replication factor and observe results.
3. Upload a large dataset into HDFS.
4. Analyze HDFS storage utilization.

Output:



Name	Date modified	Type	Size
container-executor	24-03-2026 22:44	File	755 KB
hadoop	24-03-2026 22:27	File	9 KB
hadoop	24-03-2026 22:27	Windows Comma...	11 KB
hadoop.dll	04-08-2026 22:40	Application extens...	77 KB
hdfs	24-03-2026 22:29	File	12 KB
hdfs	24-03-2026 22:29	Windows Comma...	9 KB
mapred	24-03-2026 22:45	File	7 KB
mapred	24-03-2026 22:45	Windows Comma...	7 KB
oom-listener	24-03-2026 22:44	File	32 KB
test-container-executor	24-03-2026 22:44	File	797 KB
winutils	04-08-2026 22:40	Application	117 KB
yarn	24-03-2026 22:44	File	14 KB
yarn	24-03-2026 22:44	Windows Comma...	14 KB

Key Skills to be Addressed

- Hadoop Installation and Configuration
- Environment Variable Configuration D25DCS159
- HDFS Administration Big Data Analytics (CSUE301) Addressed
- Distributed Storage Management
- Hadoop Cluster Monitoring

Tools /Technology

Windows 10 / Windows 11

OpenJDK 11 or above

Hadoop 3.x

Winutils.exe

Command Prompt / PowerShell